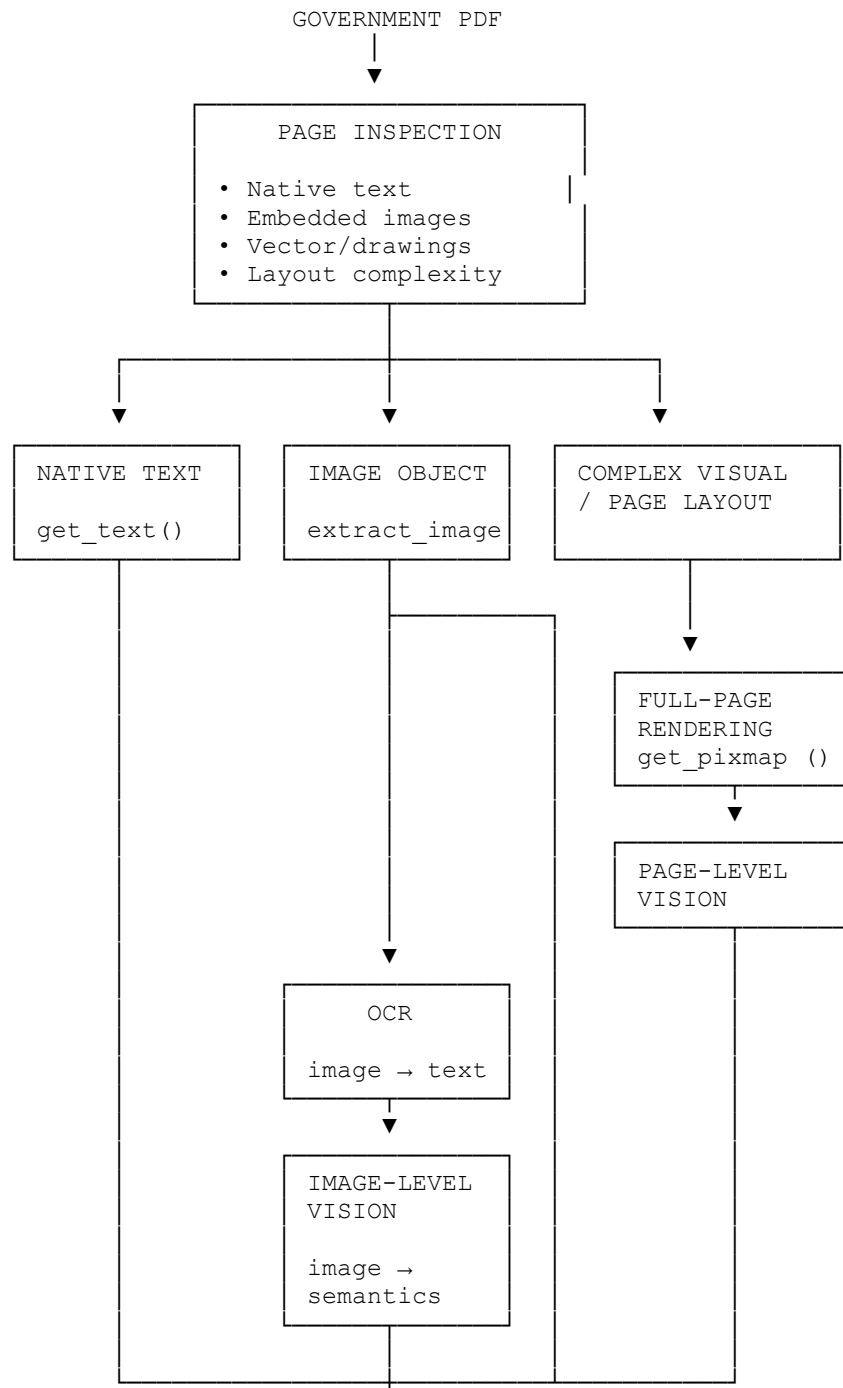
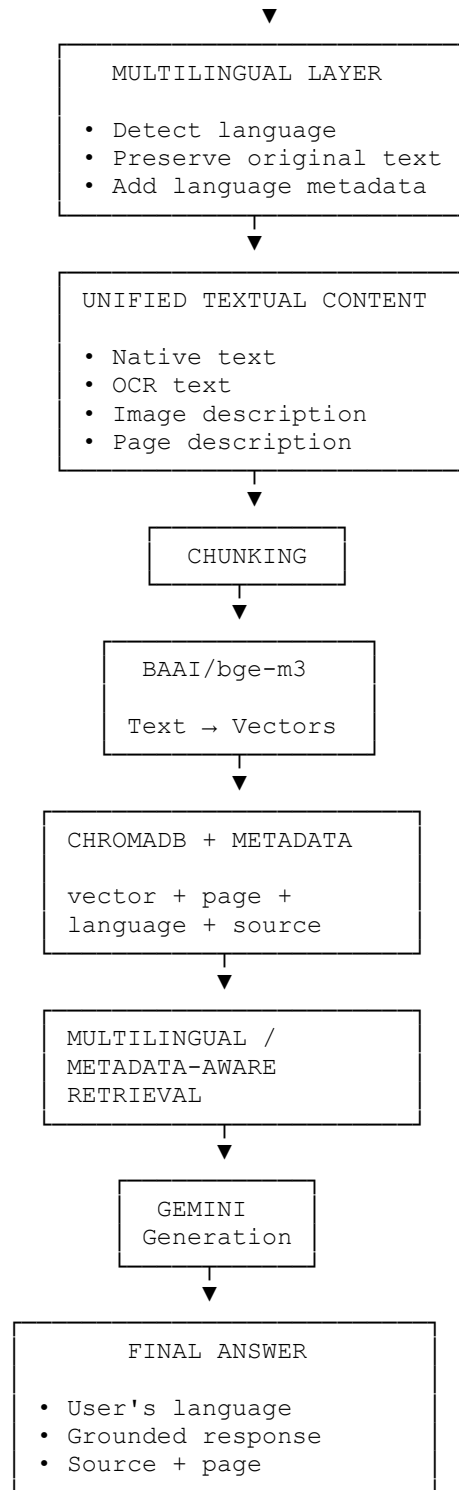


1. Complete Multimodal + Multilingual Pipeline





2. What exactly happens at each level?

Level 1 — PDF Page Inspection

We first inspect **what exists on the page**.

```
text = page.get_text().strip()
images = page.get_images(full=True)
drawings = page.get_drawings()
```

These give us three important signals:

```
text      → Does the page contain native PDF text?
images    → Does it contain embedded raster images?
drawings  → Does it contain vector graphics / shapes?
```

We don't immediately send everything to OCR or Vision.

Instead, we **route the page according to its content**.

3. Native Text

If the PDF already contains real digital text:

```
text = page.get_text().strip()
```

we preserve it directly.

Why?

Because this is generally the most direct representation of the source.

```
PDF Text Object
  ↓
page.get_text()
  ↓
Native Text
```

No OCR is necessary.

4. Embedded Image → OCR

Suppose the page contains an embedded image.

```
images = page.get_images(full=True)
```

We extract the image:

```
xref = image_info[0]
image_data = document.extract_image(xref)
```

```
image_bytes = image_data["image"]
```

Now the image bytes can be sent to OCR.

```
ocr_text = pytesseract.image_to_string(image)
```

Conceptually:

```
Embedded Image
  ↓
Extract Image
  ↓
OCR
  ↓
Text
```

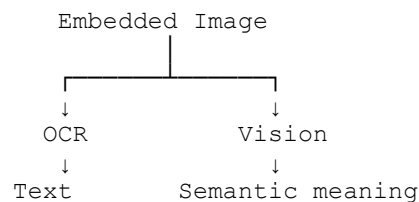
Why OCR?

Because the image may contain:

- scanned text
- scanned office orders
- photographs of documents
- screenshots containing text

5. Embedded Image → Image-Level Vision

The **same image** can also go to a Vision model.



For example:

```
Image
  ↓
OCR
  ↓
"Application shall be submitted..."
```

while Vision might produce:

```
"The image contains an official government
notice with a heading, an application table
and a signature section."
```

These are **different pieces of information**.

6. Complex Page → Full-Page Vision

This is the important distinction.

Suppose the page contains:

Government Scheme

Department	Eligibility
Ministry A	Citizens
Ministry B	Organizations

↓
Chart

The meaning depends on **relationships between objects**.

Instead of sending individual lines/shapes to Vision, we render the entire page:

```
pixmap = page.get_pixmap(dpi=150)
page_image = pixmap.tobytes("png")
```

Then:

```
Complete PDF Page
    ↓
  Rasterization
    ↓
  Page Image
    ↓
Page-Level Vision
```

The Vision model can now see:

- headings
 - tables
 - rows and columns
 - chart labels
 - diagrams
 - relative positions
 - relationships between elements
-

7. Therefore Vision Exists at Two Levels

This is how I would explain it in your report:

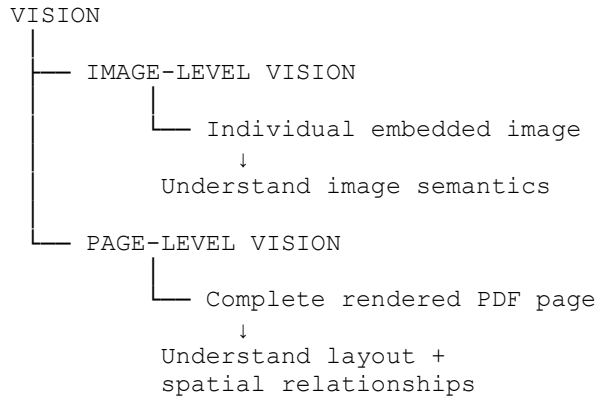


Image-level Vision asks:

What does this particular image represent?

Page-level Vision asks:

What does this entire page represent when all visual elements and their relationships are considered together?

8. Multilingual Processing

After extraction/understanding, we have textual representations:

```
Native Text
OCR Text
Image Description
Page Description
```

Now we detect language.

```
language = detect_language(text)
```

For example:

```
"Government of India..."
↓
language = en
```

or:

```
"भारत सरकार..."
```

↓
language = hi

Important

We do not translate everything into English.

Instead:

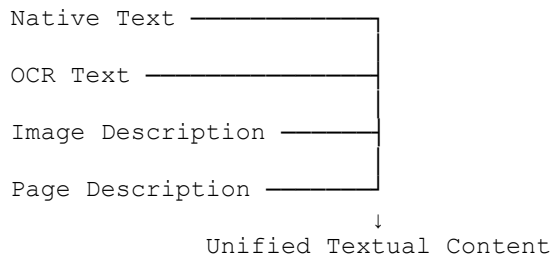
Original Content
↓
Language Detection
↓
Preserve Original Language
↓
Language Metadata

For example:

```
metadata = {  
    "language": "hi",  
    "page_number": 5,  
    "content_type": "ocr"  
}
```

9. Unified Representation

Now the different branches converge:



This is a very important architectural point.

Although our input is multimodal:

Text + Images + Visual Layout

the retrieval layer can remain unified because the information is converted into **textual semantic representations** before embedding.

10. Chunking

The unified content then enters the existing chunking stage.

```
Unified Content
  ↓
  Chunker
  ↓
Chunk 1
Chunk 2
Chunk 3
...
```

Each chunk retains metadata:

```
{
  "text": "...",
  "document_id": "...",
  "page_number": 5,
  "language": "hi",
  "content_type": "ocr"
}
```

11. BAAI/bge-m3

Now the existing embedding model from your NLP baseline comes into play.

```
Chunk
  ↓
BAAI/bge-m3
  ↓
Vector
```

For example:

```
Hindi OCR text
  ↓
BAAI/bge-m3
  ↓
[0.021, -0.183, 0.442, ...]
```

The important point is:

BAAI/bge-m3 receives textual representations, not raw images.

So:

```
Image
  ↓
OCR / Vision
  ↓
Text
  ↓
bge-m3
  ↓
Vector
```

12. ChromaDB + Metadata

The vector and metadata are stored together.

```
metadata = {  
    "document_id": document_id,  
    "document_name": document_name,  
    "page_number": page_number,  
    "language": language,  
    "content_type": content_type,  
    "text_source": text_source  
}
```

So ChromaDB contains conceptually:

VECTOR [0.021, -0.183, 0.442, ...]
METADATA document = order.pdf page = 5 language = hi content_type = ocr text_source = image

This is what allows us to later answer:

Where did this information come from?

13. Retrieval

The user asks:

"इस आदेश में नियुक्ति किसकी हुई है?"

The query itself is processed:

```
User Query  
↓  
Language Detection  
↓  
Hindi  
↓  
Embedding  
↓  
Vector Search  
↓
```

Relevant Chunks

The system can also use metadata:

```
language = hi
document = particular order
page = relevant page
content_type = ocr
```

Then the retrieved content goes to Gemini.

14. Gemini

```
User Query
+
Retrieved Context
+
Instructions
↓
Gemini
↓
Grounded Answer
```

The answer can be generated in the user's/requested language.

```
Hindi Question
↓
Hindi Answer
```

while retaining:

```
Source Document
Page Number
```

for traceability.

15. Core Implementation Code

Below is the **conceptual integrated implementation**. I would use this as the code section of the white paper.

ocr_processor.py

```
# ocr_processor.py
```

```
import io
```

```
import pymupdf
```

```

import pytesseract

from PIL import Image

def ocr_image(image_bytes):
    """
    Perform OCR on an image represented as raw bytes.

    Input:
        image_bytes -> PNG/JPEG/etc. image bytes

    Output:
        Extracted text as a string
    """

    # Safety check: do not process empty image data.
    if not image_bytes:
        return ""

    # Convert raw bytes into a PIL image.
    image = Image.open(io.BytesIO(image_bytes))

    # Tesseract reads text present inside the image.
    text = pytesseract.image_to_string(image)

    # Remove unnecessary leading/trailing whitespace.
    return text.strip()

def render_pdf_page(pdf_path, page_number, dpi=150):
    """
    Render a complete PDF page into an image.

    page_number is zero-based because PyMuPDF
    indexes pages from 0.
    """

    document = pymupdf.open(pdf_path)

    try:

        # Get the requested PDF page.
        page = document[page_number]

        # Convert the complete page into pixels.
        pixmap = page.get_pixmap(dpi=dpi)

        # Convert rendered pixels into PNG bytes.
        return pixmap.tobytes("png")

    finally:

        # Always close the PDF.
        document.close()

```

16. page_classifier.py

```

# page_classifier.py

def classify_page(page, drawing_threshold=100):
    """
    Decide how a PDF page should be processed.

    This is a heuristic router, not a perfect
    semantic classifier.
    """

    # Extract native PDF text.
    text = page.get_text().strip()

    # Find embedded raster images.
    images = page.get_images(full=True)

    # Find vector drawings / paths / shapes.
    drawings = page.get_drawings()

    # -----
    # CASE 1: Simple native-text page
    # -----

    if text and not images and len(drawings) < drawing_threshold:

        return "text"

    # -----
    # CASE 2: Image/scanned-oriented page
    # -----

    if not text and images and len(drawings) < drawing_threshold:

        return "image"

    # -----
    # CASE 3: Anything more complicated
    # -----

    # This may include:
    # - text + images
    # - tables
    # - charts
    # - diagrams
    # - many vector elements
    # - mixed layouts

    return "complex"

```

17. vision_processor.py

The Vision module has two conceptual operations.

```

# vision_processor.py

import os

```

```

from dotenv import load_dotenv
from google import genai

# Load variables from .env
load_dotenv()

# Name of the environment variable containing
# the Gemini API key.
API_KEY = "Gemini_API_Key"

api_key = os.getenv(API_KEY)

# Create Gemini client.
client = genai.Client(api_key=api_key)

MODEL_NAME = "gemini-3.6-flash"

def analyze_image(image_bytes, prompt):
    """
    Analyze an individual embedded image.

    This is IMAGE-LEVEL Vision.
    """

    if not image_bytes:
        raise ValueError("Image data is empty")

    if not prompt:
        raise ValueError("Prompt is empty")

    response = client.models.generate_content(
        model=MODEL_NAME,
        contents=[
            prompt,
            {
                "inline_data": {
                    "mime_type": "image/png",
                    "data": image_bytes
                }
            }
        ]
    )

    return response.text.strip()

def analyze_page(page_image_bytes, prompt):
    """
    Analyze a complete rendered PDF page.

    This is PAGE-LEVEL Vision.
    """

    if not page_image_bytes:
        raise ValueError("Page image is empty")

    response = client.models.generate_content(
        model=MODEL_NAME,

```

```

        contents=[
            prompt,
            {
                "inline_data": {
                    "mime_type": "image/png",
                    "data": page_image_bytes
                }
            }
        ]
    )

    return response.text.strip()

```

Note: the exact multimodal `google-genai` request structure should be validated against the installed SDK before treating this code as production-ready. The architecture is the important part here; the API call should be tested in your environment.

18. Integrated `pdf_loader.py`

Now we combine everything.

```

# pdf_loader.py

from pathlib import Path

import pymupdf

from ocr_processor import (
    ocr_image,
    render_pdf_page
)

from page_classifier import classify_page

from vision_processor import (
    analyze_image,
    analyze_page
)

def load_pdf(pdf_path):

    # Open the PDF.
    document = pymupdf.open(pdf_path)

    # Document-level metadata.
    document_name = Path(pdf_path).name
    document_id = Path(pdf_path).stem

    pages = []

    # Process the PDF page-by-page.
    for page_number, page in enumerate(document, start=1):

        # -----

```

```

# 1. PAGE INSPECTION
# -----

native_text = page.get_text().strip()

images = page.get_images(full=True)

drawings = page.get_drawings()

# Decide which processing path is appropriate.
page_type = classify_page(page)

# Containers for different representations.
ocr_texts = []
image_descriptions = []
page_description = ""

# -----
# 2. PROCESS EMBEDDED IMAGES
# -----

for image_info in images:

    # xref identifies the embedded PDF image.
    xref = image_info[0]

    # Extract the actual image.
    image_data = document.extract_image(xref)

    image_bytes = image_data["image"]

    # -----
    # OCR
    # -----

    extracted_text = ocr_image(image_bytes)

    if extracted_text:
        ocr_texts.append(extracted_text)

    # -----
    # IMAGE-LEVEL VISION
    # -----

    # In production, Vision should preferably be
    # routed only to images that need semantic
    # understanding, rather than every image.
    #
    # Example prompt:
    vision_prompt = (
        "Describe the meaningful visual content "
        "of this image. Focus on information useful "
        "for document retrieval."
    )

    description = analyze_image(
        image_bytes,
        vision_prompt
    )

    if description:

```

```

        image_descriptions.append(description)

# -----
# 3. PAGE-LEVEL VISION FOR COMPLEX PAGES
# -----

if page_type == "complex":

    # Render the complete PDF page.
    page_image = render_pdf_page(
        pdf_path,
        page_number - 1,
        dpi=150
    )

    page_prompt = (
        "Analyze this complete government-document page. "
        "Describe important tables, charts, diagrams, "
        "relationships, layout, and visual information "
        "that may be useful for retrieval."
    )

    page_description = analyze_page(
        page_image,
        page_prompt
    )

# -----
# 4. BUILD UNIFIED PAGE REPRESENTATION
# -----

page_record = {

    # Exact text from PDF.
    "native_text": native_text,

    # OCR-derived text.
    "ocr_text": "\n\n".join(ocr_texts),

    # Semantic descriptions of embedded images.
    "image_descriptions": "\n\n".join(
        image_descriptions
    ),

    # Semantic description of complete complex page.
    "page_description": page_description,

    # Traceability metadata.
    "page_number": page_number,
    "document_id": document_id,
    "document_name": document_name,
    "document_path": str(pdf_path),

    # Processing metadata.
    "page_type": page_type,
    "has_images": bool(images),
    "image_count": len(images),
    "drawing_count": len(drawings)
}

pages.append(page_record)

```

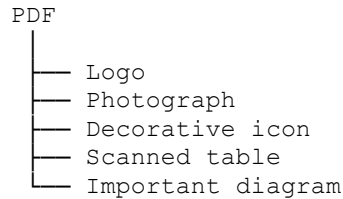


```
# Close PDF.  
document.close()  
  
return pages
```

19. One Important Improvement Before We Actually Implement This

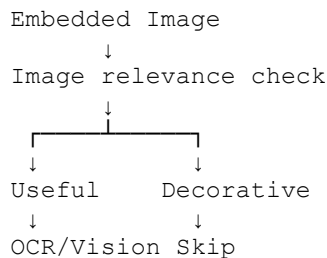
The above code demonstrates the **complete architecture**, but I would **not immediately run Image-Level Vision on every embedded image**.

For example:



Running Vision on all five is wasteful.

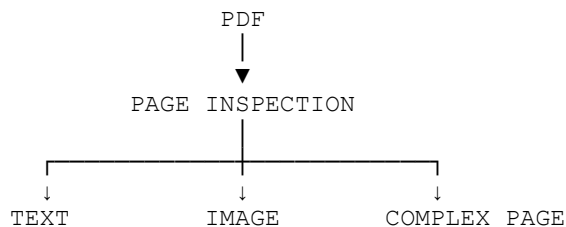
Instead, eventually:

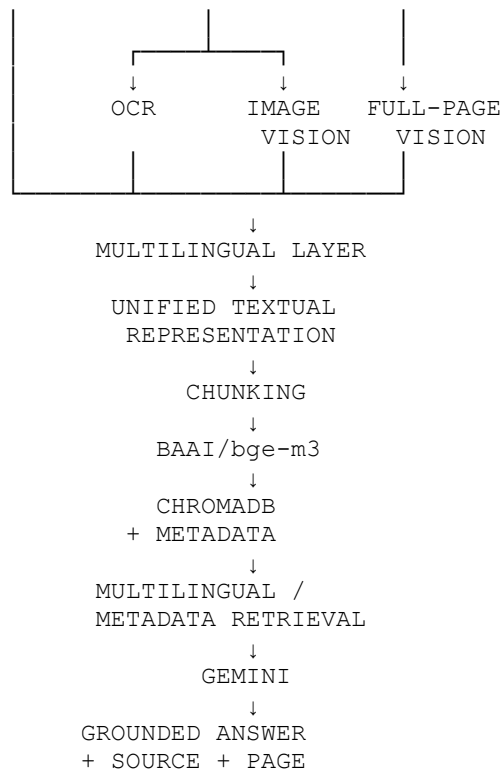


That is an optimization we can introduce after the basic pipeline works.

20. The Most Important Mental Model

I would put this exact conceptual summary in your white paper:





That is the architecture I recommend you implement. It clearly shows that **OCR, Image-Level Vision, and Page-Level Vision are different processing levels**, while everything ultimately converges into the existing RAG pipeline.

I would add this section right after the explanation of the three Vision/extraction levels:

Example: The Same Information Represented in Different Ways

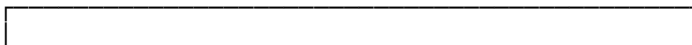
Consider the same government information:

“The application must be submitted to the District Office before 30 September 2026.”

The information may appear in a PDF in **three technically different forms**.

Case 1 — The information is native PDF text

PDF PAGE



Application Guidelines

The application must be submitted to the
District Office before 30 September 2026.

Here the sentence is an actual **PDF text object**.

PDF Text Object

↓

page.get_text()

↓

"The application must be submitted..."

Processing

Native Text

↓

Text Extraction

↓

Clean Text

↓

Language Detection

↓

Chunking

No OCR is required.

Case 2 — The same information exists inside an image

Now imagine exactly the same sentence is part of a scanned notice:

PDF PAGE

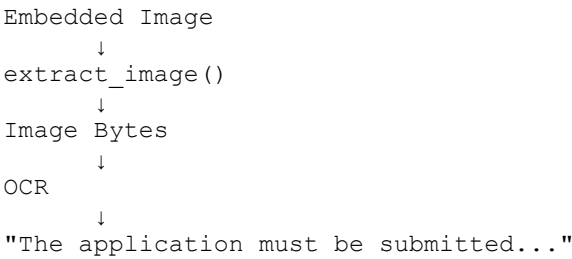
[SCANNED NOTICE]

Application Guidelines

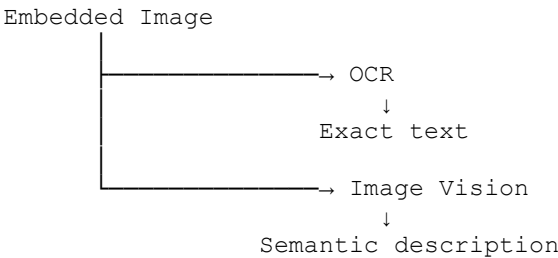
The application must be submitted
to the District Office before
30 September 2026.

The sentence **looks like text to us**, but technically it is just pixels.

Therefore:



And now Image-Level Vision can also operate on that image:



For example, Vision could produce:

“The image contains an official application notice specifying that applications must be submitted to the District Office before 30 September 2026.”

So:

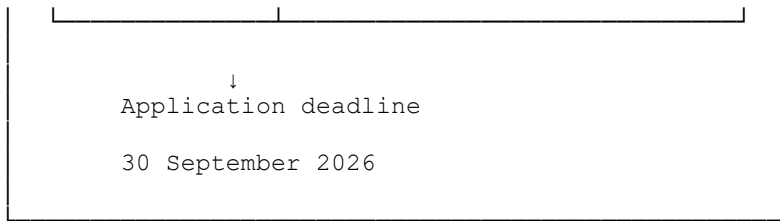
OCR → primarily recovers the text.

Image Vision → interprets the meaning/content of the image.

Case 3 — The same information is part of a complex full PDF page

Now imagine the information is presented as part of a larger government page:

GOVERNMENT SCHEME	
Application Procedure	
STEP	REQUIREMENT
1	Fill application form
2	Attach required documents
3	Submit to District Office



Here, the meaning depends not just on individual words.

It depends on:

- the heading,
- table structure,
- rows and columns,
- arrows,
- spatial positioning,
- relationship between “District Office” and the deadline.

Therefore we render the **complete page**:

Complete PDF Page

↓
get_pixmap()

↓
Full-page image

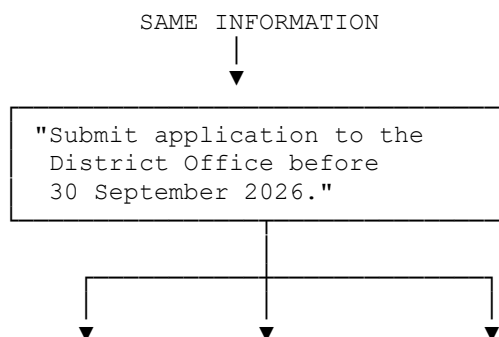
↓
PAGE-LEVEL VISION

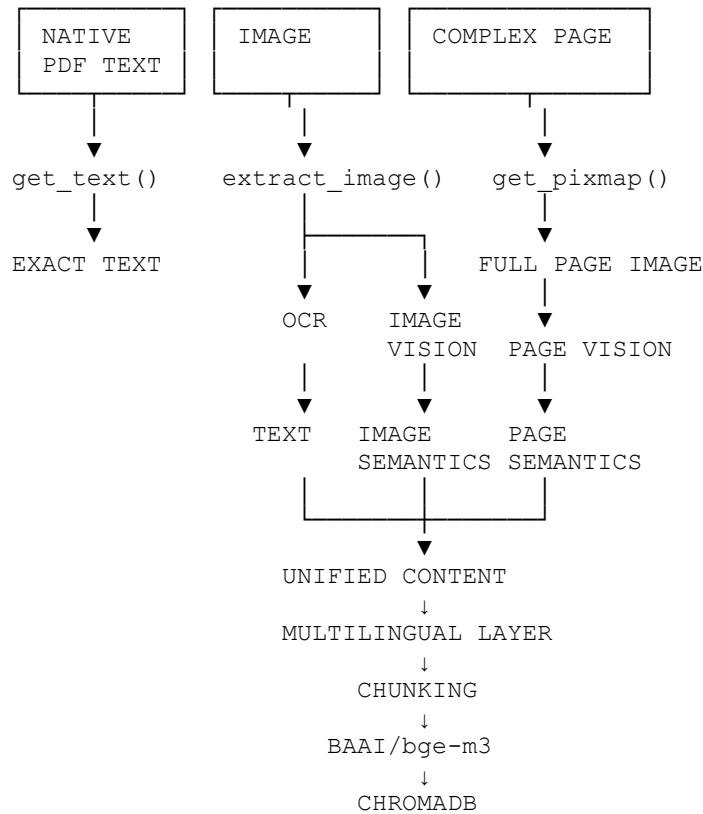
↓
Understand:

- table
- heading
- relationships
- layout
- deadline
- overall meaning

The Three Cases Side-by-Side

This is probably the **most useful diagram to include in your white paper**:





The fundamental distinction

Native Text



"What exactly does the PDF text say?"

Image OCR



"What text is written inside this image?"

Image-Level Vision



"What does this individual image represent?"

Page-Level Vision



"What does this complete page mean when considering its visual/layout relationships?"

And importantly, **the same information can ultimately reach the RAG system through all these different routes**, depending on how that information was physically represented in the original PDF.

That is exactly why your project is **multimodal** rather than simply an OCR-based RAG system.

